Programming with Generative AI

Viraj Kumar
Kotak-IISc AI-ML Centre

Lecture 11:

- Advanced iteration and data structures
- High-quality Tests



A CSR Initiative by



NPTEL

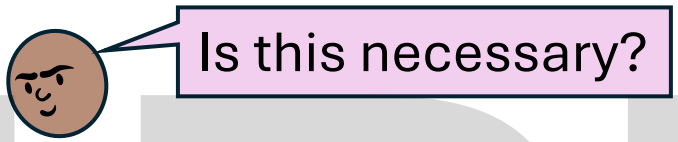
What is this recursive function's purpose?

```
def mystery(n: int=3) -> str:
    while n > 0:
        result = input()
        if result:
            return result
        n -= 1
    return ''
```

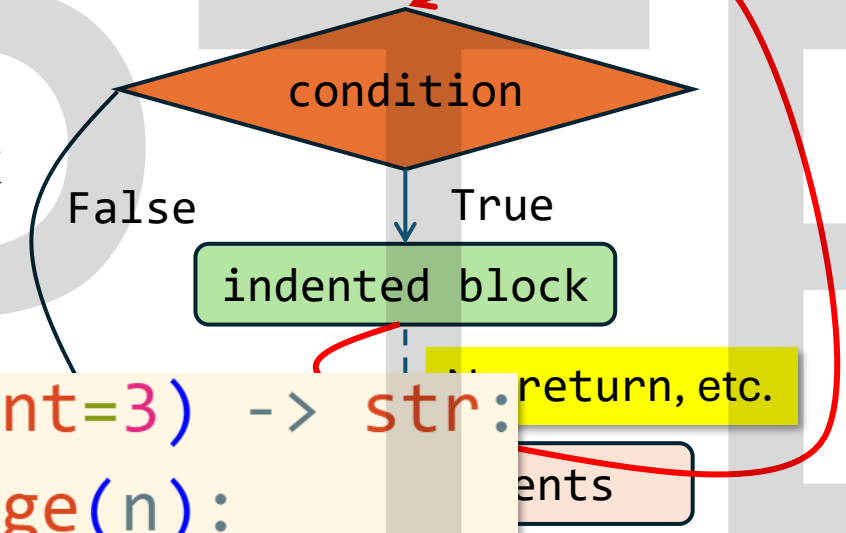
Parameter n has a **default argument** (3)

PythonTutor

Repeated `if` = `while`




```
while condition:
    indented block
    of statements
other statements
```




```
def mystery(n: int=3) -> str:
    for _ in range(n):
        result = input()
        if result:
            return result
    return ''
```

Replace `while`-loop with `for`-loop (1/2)

```
def frequency(data: list, object) -> int:
    result = 0
    for item in data:
        if item is object:
            result += 1
    return result
```

NPTEL

Replace `while`-loop with `for`-loop (2/2)

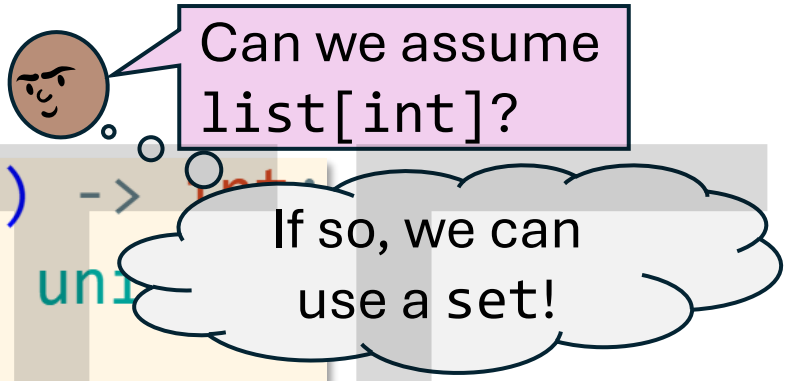
```
def indices(data: list, object) -> list[int]:
    return [i for i, item in enumerate(data) if item is object]
    for i, item in enumerate(data):
        if item is object:
            result.append(i)
    return result
        i += 1
    return result
```

Searching in a sorted list

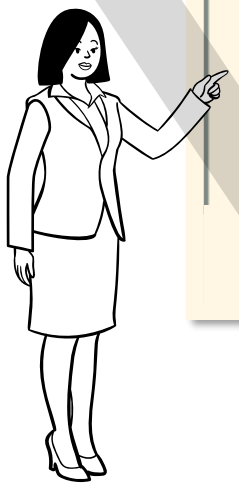
```
def binary_search(data: list[int], key: int) -> bool:
    """Binary search for a key in a sorted list."""
    low = 0
    high = len(data) - 1
    while low <= high:
        mid = (low + high) // 2
        if data[mid] < key:
            low = mid + 1
        elif data[mid] > key:
            high = mid - 1
        else:
            return True
    return False
```

NPTEL

Ask Clarifying Questions




```
def num_unique(data: list) -> int:
    """Find the number of unique
    items in data.
    >>> num_unique([1, 2, 1])
    2
    """
```



The set data structure

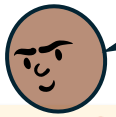
```
>>> data = [1, 2, 1]
>>> s = set(data)
```

Most of Python's immutable built-in objects are hashable; mutable containers (such as lists or dictionaries) are not; immutable containers (such as tuples and frozensets) are only hashable if their elements are hashable.

```
>>> s = {'a'}
>>> s.add(2)
>>> s
{2, 'a'}
>>> s.add([])
Traceback (most recent call last):
TypeError: unhashable type: 'list'
```

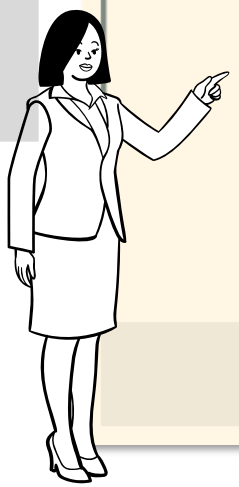
NPTEL

AI-generated suggestion 1



Can we assume `list[int]`?

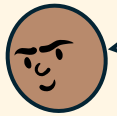
```
def num_unique(data: list) -> int:
    """Find the number of unique
    items in data.
    >>> num_unique([1, 2, 1])
    2
    >>> num_unique([[1], [1], '3'])
    2
    """
    return len(set(data))
```



AI-generated suggestion 2

```
def num_unique(data: list) -> int:
    """Find the number of unique
    >>>data = [[1], [1], '3']
    >>>data = sorted(data)
        Traceback (most recent call last):
    TypeError: '<' not supported between instances of 'str' and 'list'
    2
    """
    # Step 1: Sort the data
    data = sorted(data)
    # Step 2: Count the number of unique items
    count = 0
    for i in range(len(data) - 1):
        if data[i] != data[i + 1]:
            count += 1
```

Technical prompt



Instead, how about:
`data.sort()`

NPTEL

AI-generated suggestion 3

```
def num_unique(data: list) -> int:
    2
    >>> num_unique([[1], [1], '3'])
    2
    """
    count = 0
    # Use two nested loops
    for i in range(len(data)):
        for j in range(i + 1, len(data)):
            if data[i] == data[j]:
                break
        else:
            count += 1
```

Pattern 1: Iterate until `data[i]` found in `data[i+1:]`

Pattern 2: Accumulate the count

Versions of range	
`range(n)`	`0, 1, ..., <n`
`range(m, n)`	`m, m+1, ..., <n`
`range(m, n, k)`	`m, m+k, ..., <n`

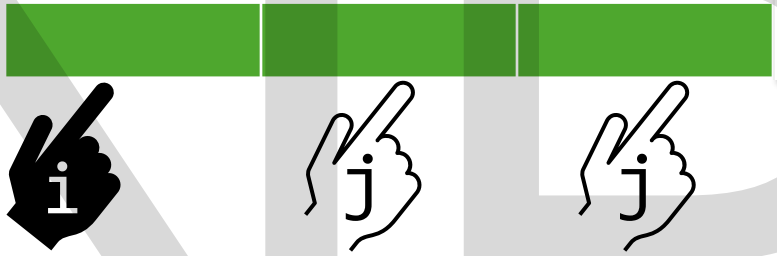
PythonTutor

NPTEL

Pattern 3: All-pairs

Explore Python's itertools module:

```
>>> from itertools import combinations
>>> data = [-7, 1, 2, 5, 16]
>>> for x, y in combinations(data, 2):
...     # do something with x, y
```




```
for i in range(len(seq)):
    for j in range(i + 1, len(seq)):
        # do something with
        # seq(i) and seq(j)
```

NPTEL

The “most important” data structure

- *Source*: NCERT’s *National Achievement Survey* (NAS 2021)

Class	Mathematics (500)
III	306
V	284
VIII	255
X	220

```
>>> maths = {
...     'III': 306,
...     'V': 284,
...     'VIII': 255,
...     'X': 220
... }
>>> type(maths)
<class 'dict'>
```

```
key: value
```

```
if maths['VIII'] < maths['X']:
    # ...
```

```
maths = [
    306, # III
    284, # V
    255, # VIII
    220  # X
]
```

```
if maths[2] < maths[3]:
    # ...
```

Basic `dict` operations

```
>>> maths
{'III': 306, 'V': 284, 'VIII': 255, 'X': 220}
>>> len(maths)
4
>>> 'V' in maths
True
>>> 284 in maths
False
```

```
>>> for k, v in maths.items()
>>> id(maths)
2223531351424
>>> maths['II'] = 'Not measured'
>>> maths['III'] += 1
>>> maths
{'III': 307, 'V': 284, 'VIII': 255, 'X': 220, 'II': 'Not measured'}
>>> id(maths)
2223531351424
```

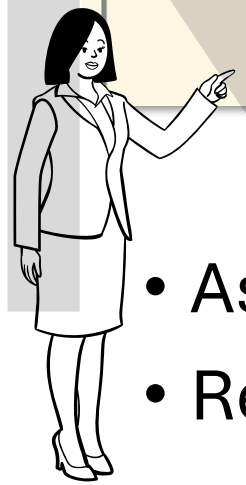
As per the **current** documentation:

Dictionaries preserve insertion order. Note that updating a key does not affect the order.

NPTEL

Ask the Client

```
def substring_count(s: str):
    """Count the number of times each
    substring of s appears in s."""
```





- What is the return type?
- How are results ordered?
- Upper vs lower case?

- Ask: CodeCheck
- Review: CodeCheck

NPTEL

Test Driven Development

- ***Before*** asking us to code, our client has created high-quality tests
- Each individual test is **valid**
  - For each test input, the test output is correct
  - If a test fails, we know that the code is buggy
- The collection of tests is **thorough**
  - If our code is buggy, it is likely to fail *some* test(s)
- Practice creating valid and thorough tests:
  - `is_prime` (CodeCheck)

```
>>> f(input)
output
```

NPTEL

Summary

- A more general form of iteration: `while`
  - Use `for` when possible
- All-pairs pattern: “slow finger, fast finger” (**itertools**)
- A **`dict`** is a mutable data structure for storing **`key`**: **`value`** pairs
  - If **`value`**: `bool`, use a **`set`**
  - If **`value`** is a count:

```
from Collections import Counter
```

- A critical skill: writing high-quality tests:
  - Each test must be **valid**
  - The collection of tests must be **thorough**

NPTEL